



## **Programación Concurrente**

### **Actividad 8**

### **“Concurrencia en Sistemas Distribuidos”**

#### **OBJETIVOS:**

- *Implementar una simulación de coordinación distribuida con quorum.*
- *Aplicar detección de fallas mediante timeouts y heartbeats.*
- *Reflexionar sobre las diferencias entre concurrencia local y sistemas distribuidos reales.*

#### **MODALIDAD**

- Trabajo grupal (mismos grupos definidos para la materia).
- Indicar los nombres de todos los integrantes en el documento.

#### **ALCANCE DEL TRABAJO**

Entregar un documento (PDF o Word) con el código Rust completo y compilable, junto con un ejemplo de salida del programa.

#### **EJERCICIO 1 — ESCRITURA CON QUORUM**

Implementar una simulación de un coordinador que replica un valor en múltiples nodos y utiliza **quorum** para decidir si la escritura fue exitosa.

##### **Escenario:**

- 1 thread **coordinador** envía un valor a **5 threads réplicas**.
- Cada réplica puede:
  - Confirmar exitosamente (con latencia variable entre 100ms y 800ms).
  - Fallar (no responder). La falla debe ser **aleatoria**: cada réplica tiene un 30% de probabilidad de fallar.
- El coordinador espera confirmaciones con un **timeout de 1 segundo**.
- La escritura se considera exitosa si al menos **3 de 5 réplicas** confirman (quorum = mayoría).
- Si no se alcanza el quorum, el coordinador reporta **falla de escritura**.

##### **Requisitos:**

- Usar `std::sync::mpsc` para la comunicación réplicas → coordinador.
- Usar `recv_timeout` para el timeout del coordinador.
- La falla de cada réplica debe decidirse aleatoriamente al iniciar el thread (usar `rand`).
- Imprimir mensajes del flujo: envío, recepción, confirmación, timeout, resultado final.
- Ejecutar la simulación **3 veces** (loop) para ver distintos resultados según las fallas aleatorias.

##### **Pista de estructura:**

```

use std::sync::mpsc;
use std::thread;
use std::time::Duration;
use rand::Rng;

const REPLICAS: usize = 5;
const QUORUM: usize = 3;
const TIMEOUT: Duration = Duration::from_secs(1);

fn main() {
    // Loop: ejecutar 3 rondas de escritura
    // En cada ronda:
    //   - Crear channel para confirmaciones
    //   - Spawn 5 threads réplicas (cada una decide
    //     aleatoriamente si falla)
    //   - Coordinador espera confirmaciones con
    //     recv_timeout
    //   - Evaluar si se alcanzó quorum
    //   - Imprimir resultado
}

```

Salida esperada (ejemplo de una ronda):

```

== Ronda 1 ==
Coordinador: enviando valor 42 a 5 réplicas...
Réplica 3: falla simulada, no respondo. 💀
Réplica 5: falla simulada, no respondo. 💀
Réplica 1: recibí valor 42, confirmando. (latencia: 230ms)
Réplica 2: recibí valor 42, confirmando. (latencia: 450ms)
Réplica 4: recibí valor 42, confirmando. (latencia: 670ms)
Coordinador: confirmación recibida de Réplica 1.
Coordinador: confirmación recibida de Réplica 2.
Coordinador: confirmación recibida de Réplica 4.
Coordinador: quorum alcanzado (3/5). Escritura EXITOSA. ✅

```

## EJERCICIO 2 — HEARTBEAT Y DETECCIÓN DE FALLAS

Implementar un sistema de monitoreo donde un monitor detecta la caída de nodos mediante heartbeats.

### Escenario:

- **3 threads nodos** que envían heartbeats periódicos (cada 500ms) a un thread **monitor**.
- El **monitor** espera heartbeats de cada nodo. Si no recibe heartbeat de un nodo en 1.5 segundos, lo declara caído.
- Después de un tiempo aleatorio (entre 2 y 4 segundos), **uno de los nodos deja de enviar heartbeats** (simula crash).
- Otro nodo puede "volverse lento" (enviar heartbeats cada 1.2 segundos en vez de 500ms) simulando una red congestionada.
- El monitor debe reportar el estado de cada nodo: ACTIVO, SOSPECHOSO (un timeout), CAÍDO (dos timeouts consecutivos).

### Requisitos:

- Usar mpsc channels para los heartbeats (cada nodo envía su ID periódicamente).
- El monitor usa recv\_timeout para detectar ausencia de heartbeats.

- Mantener un registro del último heartbeat de cada nodo.
- Distinguir entre un nodo caído y un nodo lento (al menos intentar).
- El programa debe correr por 8 segundos y luego imprimir un reporte final del estado de cada nodo.
- El programa debe terminar limpiamente (sin colgarse).

#### Pista de estructura:

```
use std::sync::mpsc;
use std::thread;
use std::time::{Duration, Instant};
use std::collections::HashMap;

enum EstadoNodo {
    Activo,
    Sospechoso,
    Caído,
}

fn main() {
    // Crear channel compartido para heartbeats (todos los nodos -> monitor)
    // Spawn 3 nodos con comportamiento distinto:
    // - Nodo 1: normal (heartbeat cada 500ms, siempre)
    // - Nodo 2: se vuelve lento después de 3s (heartbeat cada 1200ms)
    // - Nodo 3: muere después de 2-4s (deja de enviar)
    // Monitor: loop con recv_timeout, actualizar estado de nodos
    // Después de 8s: imprimir reporte y terminar
}
```

#### Salida esperada (ejemplo):

```
[0.5s] Monitor: heartbeat de Nodo 1. Estado: ACTIVO
[0.5s] Monitor: heartbeat de Nodo 2. Estado: ACTIVO
[0.5s] Monitor: heartbeat de Nodo 3. Estado: ACTIVO
...
[3.2s] Monitor: Nodo 3 sin heartbeat hace 1.5s. Estado: SOSPECHOSO ⚠️
[4.7s] Monitor: Nodo 3 sin heartbeat hace 3.0s. Estado: CAÍDO 💀
[4.0s] Monitor: Nodo 2 heartbeat tardío (1.1s). Estado: SOSPECHOSO ⚠️
[4.6s] Monitor: heartbeat de Nodo 2. Estado: ACTIVO (recuperado)
...

== Reporte final (8s) ==
Nodo 1: ACTIVO (último heartbeat hace 0.3s)
Nodo 2: ACTIVO (último heartbeat hace 0.9s) – tuvo episodios de lentitud
Nodo 3: CAÍDO (último heartbeat hace 5.2s) – declarado caído en t=4.7s
```

### EJERCICIO 3 — PREGUNTAS CONCEPTUALES

Responder las siguientes preguntas (extensión sugerida: 3-5 líneas cada una):

- ¿Por qué los ejercicios 1 y 2 NO son sistemas distribuidos reales? Mencionar al menos 3 garantías que tiene mpsc que una red real no tiene.
- Quorum y CAP: En el ejercicio 1, si bajamos el quorum a 1 (confirmar con solo 1 réplica), ¿qué

ganamos y qué perdemos en términos del teorema CAP?

- c) Falsos positivos: En el ejercicio 2, el nodo 2 se vuelve lento pero sigue vivo. En un sistema real, ¿qué consecuencias puede tener declarar "caído" a un nodo que en realidad está vivo? Dar un ejemplo concreto.
- d) El problema de los dos generales: Expliquen por qué el coordinador del ejercicio 1 nunca puede tener certeza absoluta de que una réplica recibió el valor, incluso si recibe la confirmación. Relacionar con el problema visto en clase.

## ENTREGABLES

- Documento PDF o Word con:
  - Código Rust completo (debe compilar).
  - Ejemplo de salida del programa.

## BIBLIOGRAFÍA

- *The Rust Programming Language* — Capítulo 16: Fearless Concurrency.
- Documentación de Rust: `std::sync::mpsc`, `recv_timeout`.
- Material de clase: Módulo 8 — Concurrencia en Sistemas Distribuidos.
- *Designing Data-Intensive Applications*, Martin Kleppmann — Capítulos 5 y 8.